

Algorithme de Barnes-Hut: optimisation de problèmes à N corps et arbres ternaires

La simulation d'une galaxie est une tâche complexe et demandant beaucoup de ressources de calcul mais néanmoins important pour mieux comprendre la formation de structures telles que leurs bras. Je me suis donc souvent demander de quelle manière certaines personnes arrivaient à simuler des galaxies contenant des millions d'étoiles.

Ce sujet s'ancre dans le thème de l'année en employant de nombreuses boucles algorithmiques et simulant de nombreuses trajectoires elliptiques ou circulaires.

Le candidat atteste avoir travaillé en monôme.

Positionnement thématique (ÉTAPE 1) :

- INFORMATIQUE (*Informatique pratique*)
- INFORMATIQUE (*Informatique Théorique*)
- INFORMATIQUE (*Informatique pratique*)

Mots-clés (ÉTAPE 1) :

Mots-clés (en français) Mots-clés (en anglais)

<i>Problème à N corps</i>	<i>N-Body problem</i>
<i>Simulation</i>	<i>Simulation</i>
<i>Optimisation</i>	<i>Optimisation</i>
<i>Arbre Ternaire</i>	<i>Quad-Tree</i>
<i>Parallélisation</i>	<i>Parallelisation</i>

Bibliographie commentée

Un problème à N corps est un type de problème visant à étudier l'évolution d'un système comportant N corps en interactions constantes. Cette appellation est généralement utilisée pour qualifier des systèmes de mécanique céleste (systèmes solaires, galaxies...) mais peut être généralisé à des corps liés par des potentiels quelconques. Pour des systèmes à 2 et 3 corps, des solutions analytiques existent, mais au-delà on est obligé d'opter pour une résolution numérique.

L'algorithme le plus simple consiste pour chacune des N particule à calculer les forces exercées par les $N-1$ autres particules et à appliquer une méthode d'Euler [1]. Cette méthode bien que très efficace donne une complexité asymptotique en $O(N^2)$ ce qui se prête mal à de larges systèmes comme des galaxies. L'algorithme de Barnes-Hut se place alors comme un algorithme d'approximation permettant une résolution asymptotiquement en $O(N \log N)$ [2, 3].

Pour ce faire, un traitement est réalisé construisant un Quad-tree partitionnant l'espace récursivement en 4 de telle sorte que chaque case ne contienne qu'une seule particule. Ensuite lors du calcul des forces, pour chaque particule, le Quad-tree est parcouru et pour chaque nœud un test est effectué entre le rapport la taille de la région et de la distance du barycentre des particules dans cette région à l'étoile avec une paramètre θ d'"ouverture angulaire". Si ce rapport est plus grand que θ alors le barycentre est utilisé pour le calcul de la force et les particules filles de ce nœud sont ignorées [1,2,4]. Le paramètre θ sert alors d'indicateur de précision avec $\theta = 0$ correspondant à l'algorithme naïf.

Cet algorithme offre ainsi une grande amélioration par rapport au naïf, mais il nécessite toujours de traiter chaque particule un à un ce qui reste très lent. Remarquant que chaque calcul de force est indépendant, on peut essayer d'exécuter ces calculs en même temps par exemple en exploitant la structure d'une carte graphique [5].

Pour réaliser cela, on utiliserait l'API CUDA permettant de distribuer des calculs sur une carte graphique pour réaliser cette mise en parallèle [5,6,7] mais changer de méthode nécessite une révision de l'agencement en mémoire du Quad-tree pour l'adapter à l'architecture d'un GPU mais aussi pour éviter les limitations du transfert entre le CPU et le GPU[5, 6].

Problématique retenue

Comment implémenter l'algorithme de Barnes-Hut pour optimiser une simulation de galaxie?

Quel amélioration de temps de calcul peut-on observer?

Objectifs du TIPE du candidat

- Implémenter l'algorithme naïf
- Implémenter l'algorithme de Barnes-Hut sur le processeur et sur le GPU
- Comparer les temps de calculs en fonction des paramètres d'approximation.

Références bibliographiques (ÉTAPE 1)

[1] J.DUBINSKI : A parallel tree code : *New Astronomy* , 1966, volume 1 pages 133 à 147

- [2] J.BARNES, P.HUT : A Hierarchical $O(N \log N)$ force-calculation algorithm : *Nature*, 1986, Volume 327, pages 446 à 449
- [3] S.PFALZNER, P.GIBBON : Many-Body tree methods in physics : *Cambridge university press*, 1996, pages 18 à 25
- [4] T.VENTIMIGLIA, K.WAYNE : Algorithme de Barnes-Hut : <https://arborjs.org/docs/barnes-hut>
- [5] M.BURTSCHER, K.PINGALI : An efficient CUDA implementation of the tree-based Barnes-Hut N-body algorithm : *GPU Computing Gems Emerald Edition*, 2011, pages 75 à 92
- [6] NVIDIA : Parallélisation : <https://developer.nvidia.com/gpugems/gpugems3/part-v-physics-simulation/chapter-31-fast-n-body-simulation-cuda>
- [7] NVIDIA : Utilisation de CUDA : <https://docs.nvidia.com/cuda/cuda-programming-guide/>